LABS 10 Analytics

Paul Chen | DS1001 | Professor Wright | November 5, 2026

Questions

1. Rows removed. 126 rows were dropped: 10,178 original rows minus 10,052 complete rows.

2. inplace=True. It changes model_data directly rather than returning a new DataFrame. The call returns None, so its result should not be assigned back to model_data.

3. Collapsing categories. Combining rare categories reduces sparse dummy columns and simplifies distance comparisons. A country is kept only if its count is >100; a language only if its count is >10. Counts of 100 or fewer and 10 or fewer, respectively, become “other.”

4. Target. The target is score, recoded as high or low. A target is the outcome the model is trying to predict.

5. Score threshold. I chose 65: scores >65 are high; scores ≤65 are low. LABS-7 Figure 3 peaks around the mid-to-high 60s, and the median is 65. This gives 5,001 high and 5,051 low movies after cleaning, so both classes are well represented.

```
(PaulChen0508_env) @PaulChen0508 → /workspaces/DS1001-LABS-Projects (main) $ python - <<'PY'
import pandas as pd
data = pd.read_csv("data/raw/imdb_movies.csv").dropna()
raw_score = data["score"].copy()
score_threshold = 65
data["score"] = raw_score.apply(
    lambda x: "high" if x > score_threshold else "low")
print("GitHub: PaulChen0508 | score threshold = 65")
print(pd.DataFrame({"Raw score": raw_score,
                    "Class": data["score"]}).head().to_string(index=False))
print(data["score"].value_counts().to_string())
PY
GitHub: PaulChen0508 | score threshold = 65
 Raw score Class
      73.0  high
      78.0  high
      76.0  high
      70.0  high
      61.0   low
score
low     5051
high    5001
```

Fig. 1. Correct score recoding at a threshold of 65 in the GitHub Codespace.

6. Features. The features are budget_x, revenue, country, top_genre, top_lang, and year. Features are the inputs used to predict the target.

7. Numeric and dummy variables. kNN needs numerical values to calculate distances, such as Euclidean distance. Dummy variables represent categories as 0/1 without imposing an arbitrary ranking on them.

8. Train/test split. The split uses 80% for training (8,041 movies) and 20% for testing (2,011). It returns four datasets: features_train supplies training inputs; target_train supplies their known labels; features_test supplies held-out prediction inputs; target_test supplies the true labels used to check accuracy. Technically, the first two feature objects are DataFrames and the two target objects are Series.

9. Two preparation tasks. Missing-value removal: rows with NaNs were dropped because kNN cannot calculate the required distances from missing inputs. Numeric scaling: budget, revenue, and year were scaled using training-set minima and maxima so large monetary values do not dominate distances merely because of their units.

Questions continued

10. Comparing five k values. Figure 2 reports the results on the same test set, using random_state=45.

Movie score classification: five k values

k neighbors	Test accuracy	Correct / test movies
1	64.74%	1,302/2,011
3	68.22%	1,372/2,011
5	69.42%	1,396/2,011
7	69.77%	1,403/2,011
11	71.01%	1,428/2,011

Fig. 2. Test accuracies calculated from the movie data for five values of k.

I would choose k = 11: it had the highest test accuracy, 71.01%, among the five values tried. This exceeds the 50.87% largest-class test baseline. This is a choice for this split, not proof that it will always perform best.

11. Copilot question and verification.

Prompt: In LABS-10, why should the score-recoding lambda compare x > score_threshold instead of model_data['score'][x] > score_threshold? Answer in English in at most 45 words. Do not edit files.

Response: Hello Data Science Student! x is the current numeric score supplied to the lambda. model_data['score'][x] treats that score as a row index, retrieving an unrelated value or causing an indexing error, instead of comparing the current score with the threshold.

Verification: Yes. apply passes each score as x, so x > 65 checks that movie's score. Series[x] instead looks up an index label. The raw-score preview and the all-row assertion verify that high means >65 and low means ≤65.

Takeaway

Prepare data before trusting predictions – This lab showed me that a useful model depends on how its data are prepared. Choosing a score threshold, handling missing values, and encoding features determine what kNN can compare. Testing different k values showed me why I should evaluate predictions instead of assuming the default model is best. This connects to the course’s data science pipeline, where preparation, modeling, and interpretation work together. For my final project, I will document these decisions and check results before drawing conclusions.

Notes

These notes are excluded from the three-page report limit.

Reproduce the analysis

- Work in PaulChen0508/DS1001-LABS-Projects. Select PaulChen0508_env as the notebook kernel and activate the same environment in the terminal.
- Activate: source PaulChen0508_env/bin/activate. Install: pip install scikit-learn. Record the environment: pip freeze > requirements.txt.
- Raw data path: data/raw/imdb_movies.csv. Run the notebook from the project root. Restart the kernel and run all cells from the top after changes; do not recode already-recoded score labels.
- Workflow: inspect → copy → drop missing rows → collapse rare categories → simplify genre/language and dates → recode score → separate features/target → dummy encode → split → scale → create/fit/predict/evaluate.
- The score lambda receives a score value as x. Compare x > score_threshold directly. model_data["score"][x] incorrectly treats the score as a row index.
- Fit MinMaxScaler on features_train only, then transform features_test using that fitted scaler. The cleaned CSV preserves original numeric units; scaling is performed in the model feature matrices.
- Keep the same split when comparing k values. For a later project, use validation or cross-validation to choose k and a separate final test set to report performance.
- Zero scores remain in this assignment because their meaning is not documented as missing. For a later analysis, investigate them before deciding how to handle them. Revenue is an observed box-office result; a model intended for predictions before release would need features available at that time.
- Outputs: results/lab10/imdb_movies_cleaned.csv, knn_accuracy_results.csv, score_threshold_check.csv, and analysis_summary.json. Figure: figures/lab10/k_accuracy_table.png.
- Save the notebook and required files, review git status, then add, commit with a descriptive message, and push. Keep unrelated changes out of the lab commit.

Sources and checks

Dataset: Ashpal Singh, IMDB movies dataset (Kaggle)

The downloaded CSV is unchanged: 10,178 rows and 12 columns. It matches the raw CSV used for LABS-7. Score distribution reference: Paul Chen, LABS-7 Design, question 9, Figure 3.

- pandas DataFrame.dropna
- scikit-learn train_test_split
- scikit-learn MinMaxScaler
- scikit-learn KNeighborsClassifier

Run versions: Python 3.14.2; pandas 3.0.6; scikit-learn 1.9.1. Split seed: 45. Raw CSV SHA-256: acefbb7fb126eb9365b8ba2121d2c1d98cd1ede7c759b08d2315d69cfffd6ee1.

AI assistance: Codex assisted with the code, checks, and report preparation. The response in Question 11 was obtained from Copilot inside the Codespace.